# Geth Snapshot 명령어 상세 분석

`geth snapshot` 명령어는 Ethereum 상태 스냅샷을 관리하고 검증하는 고급 도구입니다.

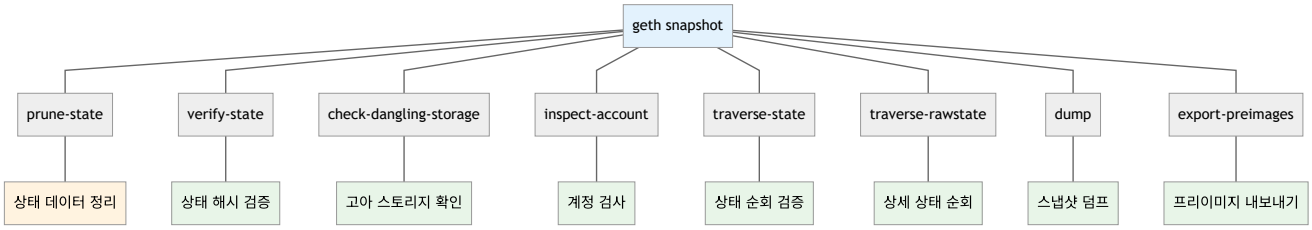
> **명령어:** `geth snapshot <subcommand>`

> **진입점:** `snapshotCommand` → 각 서브명령어 함수

> **소스 파일:** `cmd/geth/snapshot.go`

> **카테고리:** 상태 관리

## 🚀 전체 명령어 구조




## 📋 명령어 정의 및 구조

### 메인 Snapshot 명령어

```
// cmd/geth/snapshot.go

snapshotCommand = &cli.Command{
```

```
Name: "snapshot",

Usage: "A set of commands based on the snapshot",

Description: "",

Subcommands: []*cli.Command{

// 8개의 서브명령어들...

},

}
```

## 📋 서브명령어별 상세 분석

### 1. `geth snapshot prune-state` - 상태 데이터 정리

```
{

Name: "prune-state",

Usage: "Prune stale ethereum state data based on the snapshot",

ArgsUsage: "<root>",

Action: pruneState,

Flags: slices.Concat([]cli.Flag{

utils.BloomFilterSizeFlag,

}, utils.NetworkFlags, utils.DatabaseFlags),

Description: `

geth snapshot prune-state <state-root>

will prune historical state data with the help of the state snapshot.
```

```
All trie nodes and contract codes that do not belong to the specified

version state will be deleted from the database. After pruning, only

two version states are available: genesis and the specific one.


The default pruning target is the HEAD-127 state.


WARNING: it's only supported in hash mode(--state.scheme=hash)".

`,

}


// pruneState 함수

func  pruneState(ctx *cli.Context) error {

// 1. 노드 스택 생성

stack, _ := makeConfigNode(ctx)

defer  stack.Close()

// 2. 체인 데이터베이스 열기

chaindb := utils.MakeChainDatabase(ctx, stack, false)

defer  chaindb.Close()

// 3. 상태 스킴 확인 (해시 모드만 지원)

if  rawdb.ReadStateScheme(chaindb) != rawdb.HashScheme {

log.Crit("Offline pruning is not required for path scheme")

}

// 4. 프루너 설정
```

```
prunerconfig := pruner.Config{
Datadir: stack.ResolvePath(""),
BloomSize: ctx.Uint64(utils.BloomFilterSizeFlag.Name),
}
// 5. 프루너 생성
pruner, err := pruner.NewPruner(chaindb, prunerconfig)
if err != nil {
log.Error("Failed to open snapshot tree", "err", err)
return err
}
// 6. 타겟 루트 파싱
var targetRoot common.Hash
if ctx.NArg() == 1 {
targetRoot, err = parseRoot(ctx.Args().First())
if err != nil {
log.Error("Failed to resolve state root", "err", err)
return err
}
} else {
// 기본값: HEAD-127 블록
headBlock := rawdb.ReadHeadBlock(chaindb)
if headBlock == nil {
log.Error("Failed to load head block")
```

```
return errors.New("no head block")
}
targetNumber := headBlock.NumberU64()
if targetNumber > 127 {
targetNumber -= 127
} else {
targetNumber = 0
}
targetHash := rawdb.ReadCanonicalHash(chaindb, targetNumber)
targetHeader := rawdb.ReadHeader(chaindb, targetHash, targetNumber)
if targetHeader == nil {
log.Error("Failed to load target header")
return errors.New("target header not found")
}
targetRoot = targetHeader.Root
}
// 7. 상태 정리 실행
log.Info("Starting state pruning", "root", targetRoot)
start := time.Now()
if err := pruner.Prune(targetRoot); err != nil {
log.Error("Failed to prune state", "err", err)
return err
}
```

```
log.Info("State pruning completed", "elapsed", time.Since(start))
return nil
}
```

**사용 예시:**

```
# 기본 정리 (HEAD-127 블록)
$ geth snapshot prune-state
INFO [날짜] Starting state pruning root=0x1234567890abcdef...
INFO [날짜] Pruning state data...
INFO [날짜] Deleted 1,234,567 trie nodes
INFO [날짜] Deleted 89,012 contract codes
INFO [날짜] State pruning completed elapsed=2h15m30s

# 특정 상태 루트로 정리
$ geth snapshot prune-state 0x1234567890abcdef1234567890abcdef1234567890abcd
INFO [날짜] Starting state pruning root=0x1234567890abcdef...
INFO [날짜] State pruning completed elapsed=1h45m20s

# 블룸 필터 크기 조정
$ geth snapshot prune-state --bloomfilter.size 2048
```

## 2. `geth snapshot verify-state` - 상태 해시 검증

```
{
Name: "verify-state",
Usage: "Recalculate state hash based on the snapshot for verification",
ArgsUsage: "<root>",
Action: verifyState,
Flags: slices.Concat(utils.NetworkFlags, utils.DatabaseFlags),
Description: `
geth snapshot verify-state <state-root>
will traverse the whole accounts and storages set based on the specified
snapshot and recalculate the root hash of state for verification.
In other words, this command does the snapshot to trie conversion.
`,
}


// verifyState 함수
func verifyState(ctx *cli.Context) error {
// 1. 인수 검증
if ctx.NArg() != 1 {
return errors.New("need state root as the only argument")
}
// 2. 상태 루트 파싱
root, err := parseRoot(ctx.Args().First())
```

```
if err != nil {
return err
}
// 3. 데이터베이스 설정
stack, _ := makeConfigNode(ctx)
defer stack.Close()
chaindb := utils.MakeChainDatabase(ctx, stack, true) // readonly
defer chaindb.Close()
// 4. 스냅샷 트리 열기
snaptree, err := snapshot.New(snapshot.Config{
CacheSize: 256,
Recovery: false,
NoBuild: true,
AsyncBuild: false,
}, chaindb, trie.NewDatabase(chaindb), root)
if err != nil {
return err
}
defer snaptree.Release()
// 5. 상태 검증 실행
log.Info("Starting state verification", "root", root)
start := time.Now()
// 6. 계정 순회 및 검증
```

```
accountIt := snaptree.AccountIterator(root, common.Hash{})

defer accountIt.Release()

var (

accounts = 0

slots = 0

codes = 0

lastReport = time.Now()

)

for accountIt.Next() {

account := &types.StateAccount{}

if err := rlp.DecodeBytes(accountIt.Account(), account); err != nil {

return fmt.Errorf("invalid account %x: %v", accountIt.Hash(), err)

}

accounts++

// 7. 스토리지 검증

if !bytes.Equal(account.Root, types.EmptyRootHash.Bytes()) {

storageIt, err := snaptree.StorageIterator(root, accountIt.Hash(), common.Hash{}

if err != nil {

return fmt.Errorf("failed to create storage iterator: %v", err)

}

for storageIt.Next() {

slots++

}
```

```
storageIt.Release()

}

// 8. 코드 검증

if !bytes.Equal(account.CodeHash, types.EmptyCodeHash.Bytes()) {

code := rawdb.ReadCode(chaindb, common.BytesToHash(account.CodeHash))

if len(code) == 0 {

return fmt.Errorf("missing code for account %x", accountIt.Hash())

}

codes++

}

// 9. 진행률 보고

if time.Since(lastReport) > 8*time.Second {

log.Info("Verifying state", "accounts", accounts, "slots", slots, "codes", codes

lastReport = time.Now()

}

}

// 10. 최종 해시 계산 및 비교

calculatedRoot, err := snaptree.Root()

if err != nil {

return fmt.Errorf("failed to calculate root: %v", err)

}

if calculatedRoot != root {

return fmt.Errorf("state root mismatch: calculated %x, expected %x", calculated
```

```
}

log.Info("State verification completed",

"root", root,

"accounts", accounts,

"slots", slots,

"codes", codes,

"elapsed", time.Since(start))

return  nil

}
```

**사용 예시:**

```
$  geth  snapshot  verify-state  0x1234567890abcdef1234567890abcdef1234567890abc

INFO [날짜] Starting state verification root=0x1234567890abcdef...

INFO [날짜] Verifying state accounts=1000000  slots=5000000  codes=50000

INFO [날짜] Verifying state accounts=2000000  slots=10000000  codes=100000

INFO [날짜] State verification completed root=0x1234567890abcdef...  accounts=256
```

## 3. `geth snapshot check-dangling-storage` - 고아 스토리지 확인

```
{

Name: "check-dangling-storage",

Usage: "Check that there is no 'dangling' snap storage",
```

```
ArgsUsage: "<root>",

Action: checkDanglingStorage,

Flags: slices.Concat(utils.NetworkFlags, utils.DatabaseFlags),

Description: `

geth snapshot check-dangling-storage <state-root> traverses the snap storage

data, and verifies that all snapshot storage data has a corresponding account.

`,

}



// checkDanglingStorage 함수

func checkDanglingStorage(ctx *cli.Context) error {

// 1. 인수 검증

if ctx.NArg() != 1 {

return errors.New("need state root as the only argument")

}

root, err := parseRoot(ctx.Args().First())

if err != nil {

return err

}

// 2. 데이터베이스 설정

stack, _ := makeConfigNode(ctx)

defer stack.Close()

chaindb := utils.MakeChainDatabase(ctx, stack, true)
```

```
defer chaindb.Close()

// 3. 스냅샷 트리 열기

snaptree, err := snapshot.New(snapshot.Config{

CacheSize: 256,

Recovery: false,

NoBuild: true,

AsyncBuild: false,

}, chaindb, trie.NewDatabase(chaindb), root)

if err != nil {

return err

}

defer snaptree.Release()

log.Info("Checking dangling storage", "root", root)

start := time.Now()

// 4. 모든 계정 수집

accounts := make(map[common.Hash]struct{})

accountIt := snaptree.AccountIterator(root, common.Hash{})

defer accountIt.Release()

for accountIt.Next() {

accounts[accountIt.Hash()] = struct{}{}

}

log.Info("Collected accounts", "count", len(accounts))

// 5. 스토리지 데이터 확인
```

```
var (
dangling = 0
total = 0
lastReport = time.Now()
)
// 스냅샷 스토리지 순회
storageIt := rawdb.NewKeyLengthIterator(chaindb.NewIterator(rawdb.SnapshotStorag
defer storageIt.Release()
for storageIt.Next() {
key := storageIt.Key()
if len(key) != 1+2*common.HashLength {
continue
}
// 계정 해시 추출
accountHash := common.BytesToHash(key[1:33])
// 해당 계정이 존재하는지 확인
if _, exists := accounts[accountHash]; !exists {
dangling++
log.Warn("Found dangling storage", "account", accountHash, "key", common.BytesTo
}
total++
// 진행률 보고
if time.Since(lastReport) > 8*time.Second {
```

```
log.Info("Checking storage", "total", total, "dangling", dangling)

lastReport = time.Now()

}

}

// 6. 결과 출력

if  dangling > 0 {

log.Error("Found dangling storage entries", "count", dangling, "total", total)

return  fmt.Errorf("found %d dangling storage entries", dangling)

} else {

log.Info("No dangling storage found", "total", total, "elapsed", time.Since(star

}

return  nil

}
```

## 4. geth snapshot inspect-account - 계정 검사

```
{

Name: "inspect-account",

Usage: "Check all snapshot layers for the specific account",

ArgsUsage: "<address | hash>",

Action: checkAccount,

Flags: slices.Concat(utils.NetworkFlags, utils.DatabaseFlags),

Description: `
```

```
geth snapshot inspect-account <address | hash> checks all snapshot layers and pr
information about the specified address.
`,
}


// checkAccount 함수
func checkAccount(ctx *cli.Context) error {
// 1. 인수 검증
if ctx.NArg() != 1 {
return errors.New("need account address or hash as the only argument")
}
var (
hash common.Hash
addr common.Address
)
// 2. 주소 또는 해시 파싱
arg := ctx.Args().First()
if len(arg) == 40 || len(arg) == 42 {
// 주소 형식
addr = common.HexToAddress(arg)
hash = crypto.Keccak256Hash(addr.Bytes())
} else if len(arg) == 64 || len(arg) == 66 {
// 해시 형식
```

```
hash = common.HexToHash(arg)
} else {
return fmt.Errorf("invalid address or hash format: %s", arg)
}
// 3. 데이터베이스 설정
stack, _ := makeConfigNode(ctx)
defer stack.Close()
chaindb := utils.MakeChainDatabase(ctx, stack, true)
defer chaindb.Close()
// 4. 헤드 블록 가져오기
headBlock := rawdb.ReadHeadBlock(chaindb)
if headBlock == nil {
return errors.New("no head block found")
}
// 5. 스냅샷 트리 열기
snaptree, err := snapshot.New(snapshot.Config{
CacheSize: 256,
Recovery: false,
NoBuild: true,
AsyncBuild: false,
}, chaindb, trie.NewDatabase(chaindb), headBlock.Root())
if err != nil {
return err
```

```
}
defer snaptree.Release()
log.Info("Inspecting account", "address", addr, "hash", hash)
// 6. 계정 정보 조회
account, err := snaptree.Account(headBlock.Root(), hash)
if err != nil {
return fmt.Errorf("failed to retrieve account: %v", err)
}
if account == nil {
fmt.Printf("Account %s (hash: %s) not found\\n", addr.Hex(), hash.Hex())
return nil
}
// 7. 계정 정보 출력
fmt.Printf("=== Account Information ===\\n")
fmt.Printf("Address: %s\\n", addr.Hex())
fmt.Printf("Hash: %s\\n", hash.Hex())
fmt.Printf("Nonce: %d\\n", account.Nonce)
fmt.Printf("Balance: %s wei\\n", account.Balance.String())
fmt.Printf("Code Hash: %s\\n", common.BytesToHash(account.CodeHash).Hex())
fmt.Printf("Storage Root: %s\\n", common.BytesToHash(account.Root).Hex())
// 8. 코드 정보
if !bytes.Equal(account.CodeHash, types.EmptyCodeHash.Bytes()) {
code := rawdb.ReadCode(chaindb, common.BytesToHash(account.CodeHash))
```

```
fmt.Printf("Code Size: %d bytes\\n", len(code))
} else {
fmt.Printf("Code: None (EOA)\\n")
}
// 9. 스토리지 정보
if !bytes.Equal(account.Root, types.EmptyRootHash.Bytes()) {
storageIt, err := snaptree.StorageIterator(headBlock.Root(), hash, common.Hash{}
if  err != nil {
return  fmt.Errorf("failed to create storage iterator: %v", err)
}
defer  storageIt.Release()
fmt.Printf("\\n=== Storage Slots ===\\n")
count := 0
for  storageIt.Next() && count < 10 { // 처음 10개만 표시
fmt.Printf("Slot %s: %s\\n",
storageIt.Hash().Hex(),
common.BytesToHash(storageIt.Slot()).Hex())
count++
}
// 전체 슬롯 수 계산
totalSlots := count
for  storageIt.Next() {
totalSlots++
```

```
}
if  totalSlots > 10 {
fmt.Printf("... and %d more slots\\n", totalSlots-10)
}
fmt.Printf("Total storage slots: %d\\n", totalSlots)
} else {
fmt.Printf("Storage: Empty\\n")
}
return  nil
}
```

## 5. geth snapshot traverse-state - 상태 순회 검증

```
{
Name: "traverse-state",
Usage: "Traverse the state with given root hash and perform quick verification",
ArgsUsage: "<root>",
Action: traverseState,
Flags: slices.Concat(utils.NetworkFlags, utils.DatabaseFlags),
Description: `
geth snapshot traverse-state <state-root>
will traverse the whole state from the given state root and will abort if any
referenced trie node or contract code is missing. This command can be used for
```

```
state integrity verification. The default checking target is the HEAD state.


It's also usable without snapshot enabled.
`,
}


// traverseState 함수
func traverseState(ctx *cli.Context) error {
// 1. 상태 루트 결정
var root common.Hash
if ctx.NArg() == 1 {
var err error
root, err = parseRoot(ctx.Args().First())
if err != nil {
return err
}
} else {
// 기본값: HEAD 상태
stack, _ := makeConfigNode(ctx)
defer stack.Close()
chaindb := utils.MakeChainDatabase(ctx, stack, true)
defer chaindb.Close()
headBlock := rawdb.ReadHeadBlock(chaindb)
```

```
if headBlock == nil {
return errors.New("no head block found")
}
root = headBlock.Root()
}
// 2. 데이터베이스 설정
stack, _ := makeConfigNode(ctx)
defer stack.Close()
chaindb := utils.MakeChainDatabase(ctx, stack, true)
defer chaindb.Close()
triedb := trie.NewDatabase(chaindb)
log.Info("Starting state traversal", "root", root)
start := time.Now()
// 3. 상태 트라이 열기
stateTrie, err := trie.New(root, triedb)
if err != nil {
return fmt.Errorf("failed to open state trie: %v", err)
}
// 4. 상태 순회
var (
accounts = 0
slots = 0
codes = 0
```

```
lastReport = time.Now()
)
it := trie.NewIterator(stateTrie.NodeIterator(nil))
for  it.Next() {
// 5. 계정 디코딩
var  account types.StateAccount
if  err := rlp.DecodeBytes(it.Value, &account);  err != nil {
return  fmt.Errorf("invalid account at %x: %v", it.Key, err)
}
accounts++
// 6. 스토리지 트라이 확인
if  account.Root != types.EmptyRootHash {
storageTrie, err := trie.New(account.Root, triedb)
if  err != nil {
return  fmt.Errorf("failed to open storage trie for account %x: %v", it.Key, err
}
storageIt := trie.NewIterator(storageTrie.NodeIterator(nil))
for  storageIt.Next() {
slots++
}
}
// 7. 코드 확인
if  account.CodeHash != types.EmptyCodeHash {
```

```
code := rawdb.ReadCode(chaindb, account.CodeHash)
if len(code) == 0 {
return fmt.Errorf("missing code for account %x", it.Key)
}
codes++
}
// 8. 진행률 보고
if time.Since(lastReport) > 8*time.Second {
log.Info("Traversing state", "accounts", accounts, "slots", slots, "codes", code
lastReport = time.Now()
}
}
log.Info("State traversal completed",
"root", root,
"accounts", accounts,
"slots", slots,
"codes", codes,
"elapsed", time.Since(start))
return nil
}
```

## 6. geth snapshot dump - 스냅샷 덤프

```
{
Name: "dump",
Usage: "Dump a specific block from storage (same as 'geth dump' but using snapsh
ArgsUsage: "[? <blockHash> | <blockNum>]",
Action: dumpState,
Flags: slices.Concat([]cli.Flag{
utils.ExcludeCodeFlag,
utils.ExcludeStorageFlag,
utils.StartKeyFlag,
utils.DumpLimitFlag,
}, utils.NetworkFlags, utils.DatabaseFlags),
Description: `
This command is semantically equivalent to 'geth dump', but uses the snapshots
as the backend data source, making this command a lot faster.

The argument is interpreted as block number or hash. If none is provided, the la
block is used.
`,
}

// dumpState 함수
func  dumpState(ctx *cli.Context) error {
```

```
// 1. 블록 결정
var (
block *types.Block
root common.Hash
)
stack, _ := makeConfigNode(ctx)
defer stack.Close()
chaindb := utils.MakeChainDatabase(ctx, stack, true)
defer chaindb.Close()
if ctx.NArg() == 0 {
// 최신 블록 사용
block = rawdb.ReadHeadBlock(chaindb)
} else {
// 특정 블록 파싱
arg := ctx.Args().First()
if len(arg) == 64 || len(arg) == 66 {
// 해시로 해석
hash := common.HexToHash(arg)
number := rawdb.ReadHeaderNumber(chaindb, hash)
if number == nil {
return fmt.Errorf("block not found: %s", arg)
}
block = rawdb.ReadBlock(chaindb, hash, *number)
```

```
} else {
// 번호로 해석
number, err := strconv.ParseUint(arg, 10, 64)
if err != nil {
return fmt.Errorf("invalid block number: %v", err)
}
hash := rawdb.ReadCanonicalHash(chaindb, number)
block = rawdb.ReadBlock(chaindb, hash, number)
}
}
if block == nil {
return errors.New("block not found")
}
root = block.Root()
// 2. 덤프 설정
conf := &state.DumpConfig{
SkipCode: ctx.Bool(utils.ExcludeCodeFlag.Name),
SkipStorage: ctx.Bool(utils.ExcludeStorageFlag.Name),
OnlyWithAddresses: true,
Max: ctx.Uint64(utils.DumpLimitFlag.Name),
}
if ctx.IsSet(utils.StartKeyFlag.Name) {
conf.Start = hexutil.MustDecode(ctx.String(utils.StartKeyFlag.Name))
```

```
}

// 3. 스냅샷 트리 열기

snaptree, err := snapshot.New(snapshot.Config{

CacheSize: 256,

Recovery: false,

NoBuild: true,

AsyncBuild: false,

}, chaindb, trie.NewDatabase(chaindb), root)

if err != nil {

return err

}

defer snaptree.Release()

log.Info("Dumping state", "block", block.Number(), "root", root)

// 4. 스냅샷 기반 덤프 실행

dump := &state.Dump{

Root: root.Hex(),

Accounts: make(map[string]state.DumpAccount),

}

// 5. 계정 순회

accountIt := snaptree.AccountIterator(root, common.BytesToHash(conf.Start))

defer accountIt.Release()

count := uint64(0)

for accountIt.Next() && (conf.Max == 0 || count < conf.Max) {
```

```
account := &types.StateAccount{}

if err := rlp.DecodeBytes(accountIt.Account(), account); err != nil {

return fmt.Errorf("invalid account %x: %v", accountIt.Hash(), err)

}

// 6. 계정 정보 구성

dumpAccount := state.DumpAccount{

Balance: account.Balance.String(),

Nonce: account.Nonce,

Root: common.BytesToHash(account.Root).Hex(),

CodeHash: common.BytesToHash(account.CodeHash).Hex(),

Address: nil, // 주소는 역계산이 어려움

}

// 7. 코드 추가 (옵션)

if !conf.SkipCode && !bytes.Equal(account.CodeHash, types.EmptyCodeHash.Bytes())

code := rawdb.ReadCode(chaindb, common.BytesToHash(account.CodeHash))

dumpAccount.Code = hexutil.Encode(code)

}

// 8. 스토리지 추가 (옵션)

if !conf.SkipStorage && !bytes.Equal(account.Root, types.EmptyRootHash.Bytes())

dumpAccount.Storage = make(map[string]string)

storageIt, err := snaptree.StorageIterator(root, accountIt.Hash(), common.Hash{}

if err != nil {

return fmt.Errorf("failed to create storage iterator: %v", err)
```

```
}
for storageIt.Next() {
dumpAccount.Storage[storageIt.Hash().Hex()] = common.BytesToHash(storageIt.Slot(
}
storageIt.Release()
}
dump.Accounts[accountIt.Hash().Hex()] = dumpAccount
count++
}
// 9. JSON 출력
output, err := json.MarshalIndent(dump, "", " ")
if err != nil {
return err
}
fmt.Println(string(output))
return nil
}
```

## 📊 실행 예시

### 1. 상태 데이터 정리

```
$ geth snapshot prune-state
```

```
INFO [날짜] Starting state pruning root=0x1234567890abcdef...

INFO [날짜] Pruning state data...

INFO [날짜] Deleted 1,234,567 trie nodes

INFO [날짜] Deleted 89,012 contract codes

INFO [날짜] Freed 45.2 GB disk space

INFO [날짜] State pruning completed elapsed=2h15m30s
```

## 2. 상태 해시 검증

```
$ geth snapshot verify-state 0x1234567890abcdef1234567890abcdef1234567890abc

INFO [날짜] Starting state verification root=0x1234567890abcdef...

INFO [날짜] Verifying state accounts=1000000 slots=5000000 codes=50000

INFO [날짜] State verification completed root=0x1234567890abcdef... accounts=250
```

## 3. 계정 검사

```
$ geth snapshot inspect-account 0x742d35Cc6634C0532925a3b8D4C9db96c4b4d8b

=== Account Information ===

Address: 0x742d35Cc6634C0532925a3b8D4C9db96c4b4d8b

Hash: 0x1234567890abcdef1234567890abcdef1234567890abcdef1234567890abcdef

Nonce: 42

Balance: 1000000000000000000 wei

Code Hash: 0xc5d2460186f7233c927e7db2dcc703c0e500b653ca82273b7bfad8045d85a470
```

```
Storage Root: 0x56e81f171bcc55a6ff8345e692c0f86e5b48e01b996cadc001622fb5e363b4

Code: None (EOA)

Storage: Empty
```

### 4. 스냅샷 덤프

```
$ geth snapshot dump 18500000 --exclude-storage --max 10

{

"root": "0x1234567890abcdef1234567890abcdef1234567890abcdef1234567890abcdef",

"accounts": {

"0xabcdef...": {

"balance": "1000000000000000000",

"nonce": 1,

"root": "0x56e81f171bcc55a6ff8345e692c0f86e5b48e01b996cadc001622fb5e363b421",

"codeHash": "0xc5d2460186f7233c927e7db2dcc703c0e500b653ca82273b7bfad8045d85a470

}

}

}
```

## 🛠️ 문제 해결

### 일반적인 오류들

#### 1. 스냅샷이 없음

```
Fatal: Failed to open snapshot tree: snapshot not found
```

**해결**: 스냅샷 생성 대기 또는 `--snapshot=false` 로 노드 재시작

**2. 상태 스킴 불일치**

```
CRIT [날짜] Offline pruning is not required for path scheme
```

**해결**: `--state.scheme=hash` 플래그 사용

**3. 메모리 부족**

```
Fatal: runtime: out of memory
```

**해결**: 시스템 메모리 증가 또는 작업 분할

## 🔒 보안 고려사항

### 위험한 작업들

- `prune-state` : 데이터 영구 삭제
- 스냅샷 기반 작업: 스냅샷 무결성에 의존

### 안전한 사용법

1. **백업**: 중요한 작업 전 데이터베이스 백업
2. **검증**: 작업 후 상태 무결성 확인
3. **테스트**: 테스트넷에서 먼저 시도

---

**참고 자료:**

- Ethereum State Snapshots
- State Pruning
- Snapshot Format

*이 문서는 go-ethereum 소스 코드 분석을 바탕으로 작성되었습니다.*